
Algoritmos e Técnicas de Programação

Tema: Introdução a linguagem Python

Professor Wendell Bento Geraldles

Instituto Federal de Goiás
campus Luziânia
<wendell.geraldes@ifg.edu.br>



- 1 A linguagem Python
- 2 Variáveis no Python
- 3 Tipos de dados no Python
- 4 Operadores no Python
- 5 Listas no Python



A linguagem Python



O que é a linguagem Python?

História

Python é uma linguagem de programação popular. Foi criada por Guido van Rossum e lançada em 1991 (W3SCHOOLS, 2024).

O que o Python pode fazer?

- O Python pode ser usado em um servidor para criar aplicativos web.
- O Python pode ser usado junto com software para criar fluxos de trabalho.
- Python pode se conectar a sistemas de banco de dados. Ele também pode ler e modificar arquivos.
- O Python pode ser usado para manipular grandes quantidades de dados e executar cálculos matemáticos complexos.
- O Python pode ser usado para prototipagem rápida ou para desenvolvimento de software pronto para produção.



O que é a linguagem Python?

Por que usar Python?

- Python funciona em diferentes plataformas (Windows, Mac, Linux, Raspberry Pi, etc).
- Python tem uma sintaxe simples semelhante à da língua inglesa.
- Python tem uma sintaxe que permite aos desenvolvedores escrever programas com menos linhas do que algumas outras linguagens de programação.
- Python roda em um sistema interpretador, o que significa que o código pode ser executado assim que é escrito. Isso significa que a prototipação pode ser muito rápida.
- Python pode ser tratado de forma procedural, orientada a objetos ou funcional.

Bom saber

A versão principal mais recente do Python é o Python 3. Nesta aula, o Python será escrito em um editor de texto. É possível escrever Python em um sistema integrado de desenvolvimento, como Thonny, Pycharm, Netbeans ou Eclipse, que são úteis ao gerenciar projetos em Python.



Primeiros Passos

Olá mundo!

Olá mundo! é um expressão muito usada em programação para testar qualquer linguagem de programação. Para isso basta conhecer um comando de saída e digitar a mensagem para que ela apareça na tela do computador. No Python não seria diferente. O comando `print("mensagem")` serve para essa finalidade.

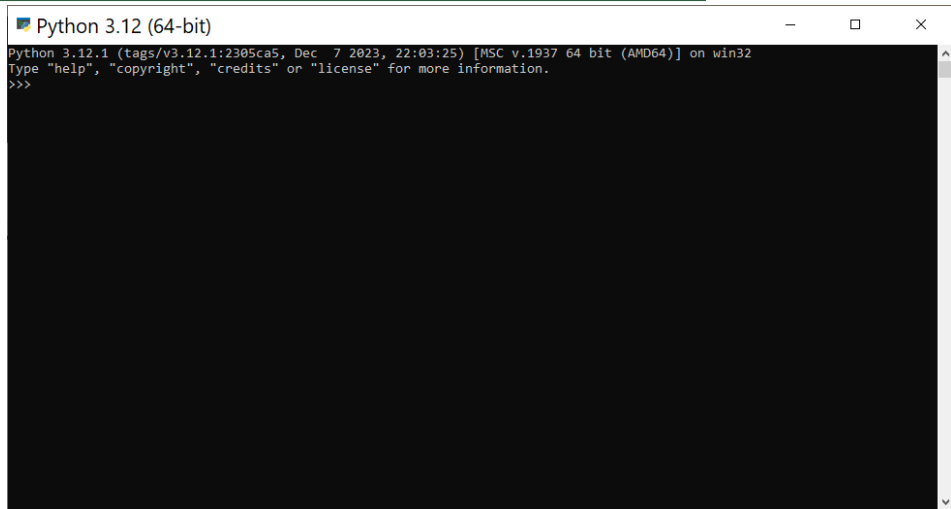
```
1 print("Ola mundo!")
```

Linha de comando ou editor de texto?

A linguagem Python possui duas maneiras de escrever código. A primeira é a linha de comando onde é possível escrever o código de forma rápida e testar seu funcionamento. A segunda é usar um editor de textos para escrever o código e depois executar o arquivo `.py`



Primeiros Passos



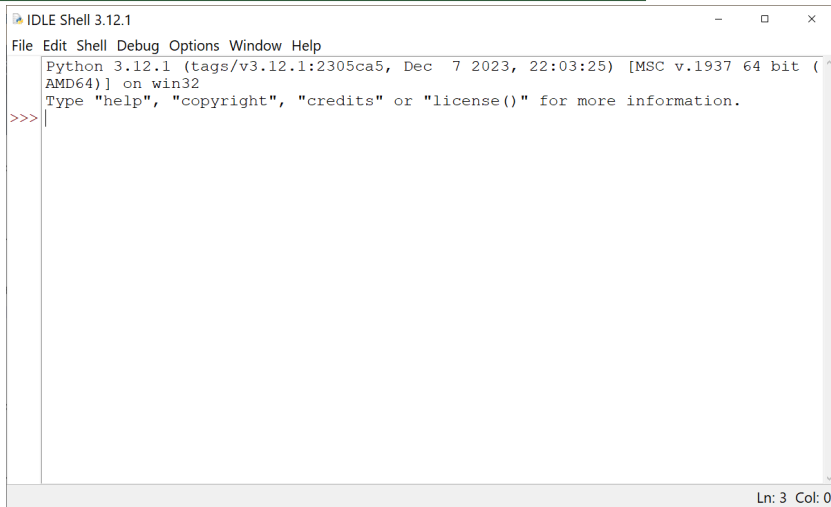
A screenshot of a Windows command prompt window titled "Python 3.12 (64-bit)". The window has standard Windows window controls (minimize, maximize, close) in the top right corner. The command prompt shows the following text:

```
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Figura: Tela da linha de comando



Primeiros Passos



The image shows a screenshot of the IDLE Shell 3.12.1 window. The window has a title bar with the text "IDLE Shell 3.12.1" and standard window controls (minimize, maximize, close). Below the title bar is a menu bar with the following items: File, Edit, Shell, Debug, Options, Window, and Help. The main area of the window is a text editor displaying the following text: "Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 64 bit (AMD64)] on win32". Below this text is a prompt ">>>" followed by a vertical cursor. The status bar at the bottom right of the window displays "Ln: 3 Col: 0".

```
IDLE Shell 3.12.1
File Edit Shell Debug Options Window Help
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> |
```

Ln: 3 Col: 0

Figura: Tela da linha de comando



Sintaxe Python

Como vimos anteriormente é possível escrever o código em Python diretamente na linha de comando.

```
>>> print("Ola mundo")  
Ola mundo  
>>>
```

Figura: Código escrito na linha de comando



Identação no Python

O recuo refere-se aos espaços no início de uma linha de código. Enquanto em outras linguagens de programação o recuo no código é apenas para legibilidade, em Python o recuo é muito importante. Python usa recuo para indicar um bloco de código.

```
2 if 5 > 2:  
3 print("Cinco é maior que dois") #Aqui vai dar erro
```

```
4 if 5 > 2:  
5     print("Cinco é maior que dois") #Aqui não vai dar erro
```



Comentários no Python

- Comentários podem ser usados para explicar o código Python.
- Comentários podem ser usados para tornar o código mais legível.
- Comentários podem ser usados para impedir a execução ao testar código.

```
6 #Aqui é uma linha de comentário  
7 print("Olá mundo!")
```

```
8 print("Olá mundo!") #Aqui é uma linha de comentário
```

```
9 #print("Olá mundo!")
```



Primeiros Passos

Comentários multilinha no Python

Para adicionar um comentário multilinha, você pode inserir um `#` para cada linha.

```
10 #Aqui é uma linha de comentário
11 #Aqui é a segunda linha de comentário
12 #Aqui é a terceira linha de comentário
13 print("Olá mundo!")
```

```
14 '''
15 Aqui é uma linha de comentário
16 Aqui é a segunda linha de comentário
17 Aqui é a terceira linha de comentário
18 '''
19 print("Olá mundo!")
```



Variáveis no Python



Variáveis no Python

Criando variáveis

O Python não tem comando para declarar uma variável. Uma variável é criada no momento em que você atribui um valor a ela pela primeira vez.

```
20 x = 5
21 y = "João"
22 print(x)
23 print(y)
```

Variáveis não precisam ser declaradas com nenhum tipo específico e podem até mesmo mudar de tipo depois de serem definidas.

```
24 x = 5 #Essa aqui é uma variável do tipo numero inteiro
25 x = "João" #Essa aqui é uma variável do tipo texto ou string
26 print(x)
```



Variáveis no Python

Criando variáveis

Se você quiser especificar o tipo de dados de uma variável, isso pode ser feito com conversão.

```
27 x = str(3) #Essa função str() converte um número em um texto ou string
28 y = int(3) #Essa função int() converte um número em um número inteiro
29 z = float(3) #Essa função float() converte um número em um número decimal
```

Você pode obter o tipo de dados de uma variável com a função `type()`.

```
30 x = 5
31 y = "João"
32 print(type(x))
33 print(type(y))
```



Variáveis no Python

Aspas simples ou aspas duplas?

Variáveis de texto ou string podem ser declaradas usando aspas simples ou duplas.

```
34 x = "João"  
35 # Você pode escrever com aspas duplas ou simples  
36 x = 'João'
```

Maiúsculas ou minúsculas?

Os nomes de variáveis são diferentes se você escrever em maiúsculas ou minúsculas.

```
37 a = 4  
38 A = "Silvia"  
39 #Neste caso a é diferente de A
```



Nomes de variáveis

Uma variável pode ter um nome curto (como `x` e `y`) ou um nome mais descritivo (`idade`, `nome`, `saldo_total`). As regras para nomes de variáveis Python são as seguintes:

- Um nome de variável deve começar com uma letra ou o caractere sublinhado
- Um nome de variável não pode começar com um número
- Um nome de variável pode conter apenas caracteres alfanuméricos e sublinhados (`Az`, `0-9` e `-`)
- Os nomes das variáveis diferenciam maiúsculas de minúsculas (`idade`, `Idade` e `IDADE` são três variáveis diferentes)
- Um nome de variável não pode ser nenhuma das palavras-chave do Python.



Variáveis no Python

Muitos valores para múltiplas variáveis

O Python permite que você atribua valores a várias variáveis em uma linha.

```
40 x, y, z = "laranja", "banana", "manga"
41 print(x)
42 print(y)
43 print(z)
```

Observação

Certifique-se de que o número de variáveis corresponde ao número de valores, caso contrário, você receberá um erro.



Variáveis no Python

Desempacotamento

Se você tem uma coleção de valores em uma lista por exemplo. O Python permite que você extraia os valores em variáveis. Isso é chamado de desempacotamento.

```
44 frutas = ["maça", "banana", "melancia"]
45 x, y, z = frutas
46 print(x)
47 print(y)
48 print(z)
```



Variáveis no Python

Variáveis de saída

A função Python, `print()` é usada para mostrar variáveis na tela do computador.

```
49 x = "Python é legal"
50 print(x)
```

Variáveis de saída

Na função `print()`, você mostra diversas variáveis, separadas por uma vírgula.

```
51 x = "Python"
52 y = "é"
53 z = "Legal"
54 print(x, y, z)
```



Variáveis no Python

Variáveis de saída

Você também pode usar o operador `+` para gerar múltiplas variáveis.

```
55 x = "Python"  
56 y = "é"  
57 z = "Legal"  
58 print(x + y + z)
```

Variáveis de saída

Na função `print()`, quando você tenta combinar uma string e um número com o operador `+`, o Python apresentará um erro.

```
59 x = 5  
60 y = "João"  
61 print(x + y)
```



Variáveis no Python

Variáveis de saída

Para números, o caractere `+` funciona como um operador matemático.

```
62 x = 5
63 y = 10
64 print(x + y)
```

Variáveis de saída

A melhor maneira de mostrar múltiplas variáveis na função `print()` é separá-las com vírgulas, isso vai mostrar diferentes tipos de dados.

```
65 x = 5
66 y = "João"
67 print(x, y)
```



Tipos de dados no Python



Tipos de dados

Na programação, o tipo de dados é um conceito importante. Variáveis podem armazenar dados de diferentes tipos, e diferentes tipos podem fazer coisas diferentes. O Python tem os seguintes tipos de dados incorporados por padrão, nestas categorias:

- Tipo de texto: `str`
- Tipos Numéricos: `int`, `float`, `complex`
- Tipos de sequência: `list`, `tuple`, `range`
- Tipo Booleano: `bool`



Tipos de dados no Python

Definindo tipos de dados

Em Python, o tipo de dado é definido quando você atribui um valor a uma variável.

```
68 x = "Olá Mundo!" #Tipo str
69 y = 10 #Tipo numerico int
70 z = 1.5 #Tipo numerico float
71 frutas = ["Maçã", "Banana", "Melancia"] #Tipo list
72 x = True #Tipo bool
```

Definindo tipos de dados específico

Se você quiser especificar o tipo de dados, poderá usar as seguintes funções construtoras.

```
73 x = str("Olá Mundo!") #Tipo str
74 x = int(20) #Tipo int
75 x = float(4.5) #Tipo float
```



Tipos de dados no Python

Números no Python

Existem três tipos numéricos em Python:

- int
- float
- complex

```
76 x = 10 #Variável do tipo int
77 y = 1.5 #variável do tipo float
78 z = 1j #variável do tipo complex
```

Para verificar o tipo de qualquer variável em Python, use a função `type()`.

```
79 print(type(x))
80 print(type(y))
81 print(type(z))
```



Tipos de dados no Python

String no Python

Strings são cadeia de caracteres (texto) e no Python elas estão sempre entre aspas duplas ou simples.

```
82 x = "Boa tarde"  
83 y = 'Boa noite'
```

Você pode exibir uma string na tela do computador usando a função `print()`.

```
84 print(x)  
85 print(y)
```



Tipos de dados no Python

Booleanos no Python

Os booleanos representam um de dois valores: True (Verdadeiro) ou False (Falso).

Valores booleanos

Na programação, muitas vezes você precisa saber se uma expressão é True ou False. Você pode avaliar qualquer expressão em Python e obter uma de duas respostas, True ou False. Quando você compara dois valores, a expressão é avaliada e o Python retorna a resposta booleana.

```
86 print(10 > 9)
87 print(10 == 9)
88 print(10 < 9)
```



Operadores no Python



O que são operadores?

Operadores são usados para executar operações em variáveis e valores. O Python divide os operadores nos seguintes grupos:

- Operadores aritméticos
- Operadores de atribuição
- Operadores de comparação
- Operadores lógicos
- Operadores de identidade
- Operadores de associação
- Operadores bit a bit



Operadores aritméticos

Operadores aritméticos são usados com valores numéricos para realizar operações matemáticas comuns.

Operador	Nome	Exemplo
+	soma	$x + y$
-	subtração	$x - y$
*	multiplicação	$x * y$
/	divisão	x / y

Tabela: Operadores aritméticos



Operadores de atribuição

Operadores de atribuição são usados para atribuir valores as variáveis.

Operador	Nome	Exemplo	Igual a
=	atribuição	x = 5	x = 5
+=	atribuição com soma	x += 5	x = x + 5
-=	atribuição com subtração	x -= y	x = x - 5
*=	atribuição com multiplicação	x *= y	x = x * 5
/=	atribuição com divisão	x /= y	x = x / 5

Tabela: Operadores de atribuição



Operadores de comparação

Operadores de comparação são usados para comparar dois valores.

Operador	Nome	Exemplo
==	igual	<code>x == y</code>
!=	não é igual	<code>x != y</code>
>	maior que	<code>x > y</code>
<	menor que	<code>x < y</code>
>=	maior e igual	<code>x >= y</code>
<=	menor e igual	<code>x <= y</code>

Tabela: Operadores de comparação



Operadores lógicos

Operadores lógicos são usados para combinar instruções condicionais.

Operador	Nome	Exemplo
and	retorna verdadeiro se ambas condições forem verdadeiras	$x > 5$ and $x < 10$
or	retorna verdadeiro se uma das condições for verdadeira	$x > 5$ or $x < 10$
not	Inverte o resultado, retorna falso se o resultado for verdadeiro	$\text{not}(x > 5 \text{ and } x < 10)$

Tabela: Operadores lógicos



Listas no Python



Listas no Python

Listas

Listas são usadas para armazenar vários itens em uma única variável. Listas são um dos quatro tipos de dados internos do Python usados para armazenar coleções de dados, os outros três são Tuple , Set e Dictionary , todos com qualidades e usos diferentes. As listas são criadas usando colchetes:

```
89 frutas = ["Maçã", "Banana", "Melancia"]  
90 print(frutas)
```

Itens da Lista

Os itens da lista são ordenados, alteráveis e permitem valores duplicados. Os itens da lista são indexados, o primeiro item tem índice [0], o segundo item tem índice [1], etc.



Listas no Python

Listas ordenadas

Quando dizemos que as listas são ordenadas, isso significa que os itens têm uma ordem definida e essa ordem não mudará. Se você adicionar novos itens a uma lista, os novos itens serão colocados no final da lista.

Listas mutáveis

A lista é mutável, o que significa que podemos alterar, adicionar e remover itens de uma lista depois que ela foi criada.

Listas duplicadas

Como as listas são indexadas, elas podem ter itens com o mesmo valor.

```
91 frutas = ["Maçã", "Banana", "Melancia", "Banana", "Melancia"]  
92 print(frutas)
```



Listas no Python

Comprimento da lista

Para determinar quantos itens uma lista possui, use a função `len()`.

```
93 frutas = ["Maçã", "Banana", "Melancia"]  
94 print(len(frutas))
```

Itens da lista - Tipo de dados

Os itens da lista podem ser de qualquer tipo de dados.

```
95 lista1 = ["Maçã", "Banana", "Melancia"]  
96 lista2 = [1, 5, 7, 9, 3]  
97 lista3 = [True, False, False]
```

A lista pode conter diferentes tipos de dados.



Listas no Python

Construtor de lista

É possível construir uma lista usando a função `list()`.

```
98 frutas = list("Maçã", "Banana", "Melancia")  
99 print(frutas)
```

Itens da lista - Tipo de dados

Os itens da lista podem ser de qualquer tipo de dados.

```
100 lista1 = ["Maçã", "Banana", "Melancia"]  
101 lista2 = [1, 5, 7, 9, 3]  
102 lista3 = [True, False, False]
```



 W3SCHOOLS. **Python**. W3C, 2024. Disponível em: <<https://www.w3schools.com>>.

